

GLG System Specification v4

Guardian Litigation Group — System Specification

README

Welcome to your Lovable project

- Project info

- How can I edit this code?

- What technologies are used for this project?

- How can I deploy this project?

- Can I connect a custom domain to my Lovable project?

Changelog

Changelog — GLG Case Management System

- Admin UX & Permissions Tightening (2026-05-28, PM)

- Phase 12 — Dialpad Integration (2026-05-28)

- Phase 11 — Integrations Management Hub (2026-05-28)

- Operation Cornerstone — Phase 7 (2026-05-28)

- Operation Cornerstone — Phase 2 (in progress, 2026-05-28)

- Operation Cornerstone — Phase 1 (2026-05-28)

Phase 1 Summary — Operation Cornerstone

- What shipped

- Database remediation (this phase)

- Linter delta

- Risks / open items carried to Phase 2

- Phase 1 sign-off

Phase 1C Summary — Tenant Feature Flags + Paralegal Lead Visibility

- Shipped

- Registered flags

- Linter delta

- Behavior verification

- Open follow-ups (Phase 1D candidates)

- Follow-up remediation (verification pass)

Phase 1D Summary — Terminology Pass

- Shipped

- Intentionally NOT changed

- Rollback

- Open items

Phase 1E Summary — DocuSeal Signed-Doc Auto-Save

- Shipped

- Out of scope (intentional)

- Risks / open items

Phase 2 Summary — Forth Adapter Hardening

- What shipped

- Files touched

- Open items / follow-ups

- Risks

- Next phase

Phase 2A Summary — PII Encryption Foundation

- What shipped

- Verification

- Out of scope (intentional)

- Risks / open items

- Phase 2A sign-off

Phase 2B Summary — Audit Log Wiring

- What shipped

- Verification

- Why DB triggers instead of app-layer wiring

- Out of scope (intentional)

- Risks / open items

- Phase 2B sign-off

Phase 2D Summary — Inactivity Session Timeout

- Goal

- Policy

- Files

- On sign-out

- Out of scope (intentional)

- Verification

Phase 2E Summary — Storage Hardening (Part 1)

- Goal

- Server-side (bucket config)

- Client-side

- Defense-in-depth

- Out of scope (intentional → Phase 2E-2)

- Verification

Phase 3 Summary — New Forth Operations

- What shipped
- Schema changes (migration applied)
- Files touched
- Open items / follow-ups
- Risks
- Next phase

Phase 4 — PLSA Adapter Abstraction (Routing Layer)

- What shipped
- Acceptance criteria
- Notes & deferred

Phase 5 — Servicing Layer Features (Complete)

- 5A — NSF Retry Logic
- 5B — Escrow Balance Automation
- 5C — Reconciliation Job + Dashboard
- 5D — Template System Extensions
- Notes / Risk

Phase 6 — Optional Phase C Extensions

- 6A — Creditor Response Tracking
- 6B — Service Graduation Automation
- Files
- Acceptance criteria
- Risks / open items

Phase 7 — Storage hardening + Realtime channel auth

- Database
- Frontend
- Backwards compatibility
- Realtime impact

Phase 11 — Integrations Management Hub

- Database
- Shared helpers — `supabase/functions/_shared/integrations.ts`
- Admin UI
- Test-connection edge functions
- Seed
- Acceptance
- Rollback (forward-only migrations — inline SQL)

Phase 12 — Dialpad Integration

- Registry + secrets
- Schema

Edge functions
Frontend
Phone matching
Acceptance
Rollback (forward-only)
Follow-ups

Guardian Litigation Group – System Specification

Version 4 · Generated 2026-05-28

README

Welcome to your Lovable project

Project info

URL: https://lovable.dev/projects/REPLACE_WITH_PROJECT_ID

How can I edit this code?

There are several ways of editing your application.

Use Lovable

Simply visit the [Lovable Project](#) and start prompting.

Changes made via Lovable will be committed automatically to this repo.

Use your preferred IDE

If you want to work locally using your own IDE, you can clone this repo and push changes. Pushed changes will also be reflected in Lovable.

The only requirement is having Node.js & npm installed - [install with nvm](#)

Follow these steps:

```
# Step 1: Clone the repository using the project's Git URL.
git clone <YOUR_GIT_URL>

# Step 2: Navigate to the project directory.
cd <YOUR_PROJECT_NAME>

# Step 3: Install the necessary dependencies.
npm i

# Step 4: Start the development server with auto-reloading and an instant preview.
npm run dev
```

Edit a file directly in GitHub

- Navigate to the desired file(s).
- Click the “Edit” button (pencil icon) at the top right of the file view.
- Make your changes and commit the changes.

Use GitHub Codespaces

- Navigate to the main page of your repository.
- Click on the “Code” button (green button) near the top right.
- Select the “Codespaces” tab.
- Click on “New codespace” to launch a new Codespace environment.
- Edit files directly within the Codespace and commit and push your changes once you’re done.

What technologies are used for this project?

This project is built with:

- Vite
- TypeScript
- React
- shadcn-ui
- Tailwind CSS

How can I deploy this project?

Simply open [Lovable](#) and click on Share -> Publish.

Can I connect a custom domain to my Lovable project?

Yes, you can!

To connect a domain, navigate to Project > Settings > Domains and click Connect Domain.

Read more here: [Setting up a custom domain](#)

Change log

Changelog – GLG Case Management System

All notable changes documented per cross-cutting rule: “Documentation by default”.

Admin UX & Permissions Tightening (2026-05-28, PM)

- **Role impersonation dropdown** added to the top bar for admins (view-only). Persists in `sessionStorage` ; client-side only — database RLS still enforces real permissions. 11 preset views (Admin, Attorney, Case Manager, Paralegal, Sales Rep, Negotiator, Payment Processor, Correspondence, Client Services Rep, Eligibility Reviewer, Viewer).
- **Leads access revoked** for `attorney` and `case_manager` roles in `role_permissions` (Client Services Rep was already off). Leads sidebar item and module now hidden for those roles. Admin / Sales Rep / Eligibility Reviewer / Negotiator (where granted) retain access. Pure paralegals still gated by `leads.paralegal_visibility` tenant flag.
- **Department field hidden** from Staff create/edit dialog. Department remains in the database (still derived from Role on save) so dashboards, staff-page grouping, and badges continue to work — it’s just no longer surfaced in the form since it was always read-only and redundant.
- **Integration test connection** buttons confirmed working for DocuSeal, Forth (Pay + CRM), and Dialpad. Each stamps `last_connected_at` / `last_connection_error` via the shared `markIntegrationConnected` helper; UI invalidates the `company-integrations` query so the “Last connected” pill refreshes immediately. No new credentials needed for any provider.

Phase 12 – Dialpad Integration (2026-05-28)

- Registered `dialpad` in `integration_providers` (telephony; events `state_changed` , `recording` , `voicemail`). Ships disabled by default per company.
- New `public.dialpad_calls` (company-scoped read, `service_role` writes). `staff` gains `dialpad_user_id` and `screen_pop_preference` .
- Edge functions: `dialpad-initiate` , `dialpad-webhook` (HMAC-SHA256 signature verification on raw body), `dialpad-backfill-user` , `dialpad-register-webhook` , `dialpad-test-connection` . All registered in `supabase/config.toml` .
- Frontend: `<CallButton>` wired into Client / Lead / Liability / Litigation / Creditor Contact surfaces; `useInitiateCall` hook; live call-state pill via realtime on `dialpad_calls` .
- `ScreenPopProvider` subscribes to inbound calls per current staff and surfaces toast/auto-navigate/off per `screen_pop_preference` .

- Webhook auto-links calls to `clients` / `leads` / `creditor_contacts` by E.164 match; completed calls append a `client_communications` row with duration and signed recording link.
- Secrets added: `DIALPAD_API_TOKEN` , `DIALPAD_WEBHOOK_SECRET` , optional `DIALPAD_DEFAULT_USER_ID` .
- See `phase_12_summary.md` for full schema, acceptance, and rollback.

Phase 11 – Integrations Management Hub (2026-05-28)

- New tables: `integration_providers` (registry, seeded `docuseal` , `forth_pay` , `forth_crm`), `company_integrations` (per-tenant config, unique `(company_id, provider_key)` , admin-only mutate), `integration_event_log` (universal observability, admin SELECT scoped by company, service_role insert).
- New admin nav item **Integrations** at `/integrations` (promoted out of Settings). Provider cards expose Configure / Test Connection / View Activity with a real-time “Last connected” pill.
- Shared edge helpers (`supabase/functions/_shared/integrations.ts`): `getIntegrationConfig` , `logIntegrationEvent` , `requireIntegrationEnabled` . Existing DocuSeal/Forth functions refactored to log + short-circuit when disabled.
- New `_shared/markIntegrationConnected.ts` stamps `last_connected_at` / `last_connection_error` on every matching provider row after a successful/failed test (handles shared-credential providers like `forth_pay` + `forth_crm`).
- Test functions: `docuseal-test` , `forth-test-connection` (forces fresh OAuth, clears ~9 day cache, stamps both Forth rows), `dialpad-test-connection` (Phase 12). UI invalidates `company-integrations` query so pills refresh immediately.
- Seed: existing tenants with DocuSeal/Forth secrets got `company_integrations` rows with `is_enabled = true` — no downtime.
- No new credentials required for Forth or DocuSeal; both reuse existing secrets.
- See `phase_11_summary.md` for full schema, acceptance, and rollback.

Operation Cornerstone – Phase 7 (2026-05-28)

7 – Storage hardening + Realtime auth

- `lead-documents` , `client-documents` , and `litigation-documents` buckets flipped to **private**. Anonymous internet read access is no longer possible.
- New SECDEF helper `public.can_access_storage_object(bucket, first_folder)` resolves the owning company by joining the first path segment to `leads` / `clients` / `litigation_matters` → `client_services` , with a fallback that treats the first segment as a company id (used by wizard scratch uploads).
- All 12 legacy `storage.objects` policies for the three buckets dropped and recreated as company-scoped `authenticated` -only policies (SELECT / INSERT / UPDATE /

DELETE).

- New frontend layer: `src/lib/storage.ts` adds `extractStoragePath()` + `getSignedDocumentUrl()` ; new `<SignedDocumentLink>` component opens a short-lived signed URL on click. Old DB rows with legacy public URLs are handled — the helper parses the path out of them.
- All four upload sites now persist the bucket-relative **path** instead of a public URL: `LeadDocumentFormDialog` , `ClientDocumentFormDialog` , `litigation/DocumentFileUpload` , and `litigation/steps/LitigationDocumentsStep` (the wizard now uploads under `{companyId}/temp/...` so RLS resolves before a matter exists).
- All three viewer surfaces switched to `<SignedDocumentLink>` : `LeadDocumentsTab` , `ClientDocumentsTab` , `LitigationDocumentList` .
- `realtime.messages` : RLS enabled with `authenticated` -only SELECT/INSERT policies. Anonymous clients can no longer subscribe to broadcast channels. Row-level filtering for `postgres_changes` continues to use the source-table RLS.

Operation Cornerstone – Phase 2 (in progress, 2026-05-28)

2E – Storage hardening (part 1)

- Bucket configs on all 4 doc buckets: 25 MB size cap + MIME allowlist (PDF, Office, images, .eml/.msg). Executables, scripts, archives, etc. rejected server-side.
- New `src/lib/storage.ts` — shared `validateDocumentUpload()` + constants wired into all 4 upload dialogs.
- 2E-2 deferred: flip public buckets to private + signed-URL read layer; virus scanning. See `phase_2e_summary.md` .

2D – Inactivity session timeout

- 30-min idle threshold with 2-min warning dialog (countdown + Stay/Sign-out actions).
- Cross-tab activity sync via `localStorage` (`glg.lastActivityAt`).
- Auto sign-out → toast → redirect to `/auth` . Only mounted inside `AppLayout` (auth pages excluded).
- New: `src/hooks/useInactivityTimeout.ts` , `src/components/auth/InactivityTimeoutDialog.tsx` . See `phase_2d_summary.md` .

2B – Audit log wiring (DB triggers)

- `audit_trigger_fn()` fires on 11 sensitive tables; `decrypt_*` helpers self-log `pii.reveal.*` . See `phase_2b_summary.md` .

2A – PII encryption foundation

- Vault key + SECDEF `encrypt_pii` / `decrypt_client_ssn` / `decrypt_lead_banking` ; new ciphertext columns. See `phase_2a_summary.md` .

Operation Cornerstone – Phase 1 (2026-05-28)

Phase 1 hardens the platform for multi-tenant commercial launch. Sub-phases were shipped in dependency order: 1B → 1A → 1C → 1E → 1D.

1B – Central audit trail log

- New table `public.system_audit_log` (`actor_user_id`, `actor_role`, `company_id`, `action`, `entity_type`, `entity_id`, `request_payload`, `response_payload`, `ip_address`, `user_agent`).
- New SECDEF function `log_audit_event(...)` callable from edge functions and frontend mutations.
- Indexes on `(actor_user_id, created_at DESC)` and `(entity_type, entity_id)` .
- RLS: staff SELECT within company scope; INSERT via SECDEF only.

1A – Security posture

- SSN encryption audit published (`docs/cornerstone/ssn_encryption_audit.md`) — current state documented; backfill deferred (logged as accepted risk).
- HIBP leaked-password protection enabled.
- Forgot-password + reset-password flows added (`/forgot-password` , `/reset-password`).
- Optional TOTP MFA opt-in card on Profile Settings.
- Input validation audit published (`docs/cornerstone/input_validation_audit.md`).
- RLS audit published (`docs/cornerstone/rls_audit_report.md`).
- API key rotation policy published (`docs/cornerstone/api_key_rotation_policy.md`).
- Rate limiting + 30-min inactivity timeout: deferred (logged as accepted risk).

1C – Multi-tenancy hardening

- New table `public.tenant_feature_flags` (`company_id`, `flag_key`, `enabled`, `value`, `description`, `updated_by`).
- New SECDEF helpers `is_feature_enabled(company_id, flag_key)` and `can_view_leads(user_id, company_id)` .
- Built-in registry: `leads.paralegal_visibility` (default OFF), `leads.show_in_navigation` (default ON).
- `leads` RLS rewritten to use `can_view_leads` ; pure paralegals gated by flag.
- Settings → Feature Flags admin tab; sidebar Leads nav respects `leads.show_in_navigation` ; `/leads` shows empty-state for gated paralegals.
- Audit event emitted on flag changes.

1E – DocuSeal signed-doc auto-save

- `docuseal-webhook` `submission.completed` handler now downloads executed PDF + audit certificate, stores them in the private `signed-documents` bucket at `{company_id}/{request_id}/{kind}-{ts}-{safe_name}` , and mirrors into `client_documents` when bound to a client.
- Archive paths appended to `signature_requests.evidence_json.archived_files` for replay.
- All archive errors caught — webhook never fails on side-effect errors.

1D — Terminology pass

- Glossary published (`docs/cornerstone/terminology_glossary.md`).
- New `src/lib/terminology.ts` constants (`TERMS`).
- UI labels swapped (display only — table names, columns, enums, routes, types unchanged):
 - “Debt Settlement” → “Consumer Debt Defense”
 - “Service” / “Services” → “Engagement” / “Engagements”
 - “Escrow” → “PLSA”
- Sweep covers: sidebar, Engagements page, Engagement form/detail/summary, Client Hub tabs + cards, Lead Conversion wizard, PLSA chart/projection, workflow + template + report field labels, docs center (schema/role/feature/roadmap guides).
- Permission `module` strings (`'Services'` in `role_permissions`) intentionally unchanged — data-driven; would require a coordinated migration.

Out of scope / deferred

- See `mem://security/deferred-cornerstone-items` for the full deferred list (SSN backfill, rate limits, inactivity timeout, lead/litigation mirroring of signed docs, backfill of previously-completed signature requests).

Phase 1 Summary – Operation Cornerstone

Window: Phase 1A (Security Hardening) + Phase 1B (Audit Log Foundation) **Status:** ☐ Complete

What shipped

1A – Security hardening

Item	Status	Artifact
HIBP leaked-password protection	☐ Enabled	<code>configure_auth(password_hibp_enabled: true)</code>
Password reset flow	☐ Built	<code>/forgot-password</code> , <code>/reset-password</code> pages + <code>reset-staff-password</code> edge function
Admin password reset (for staff)	☐ Built	<code>reset-staff-password</code> edge function (admin-gated, returns secure temp password)
MFA opt-in (TOTP)	☐ Built	<code>MfaCard</code> in Settings → Profile; uses <code>supabase.auth.mfa.enroll/challenge/verify</code>
API key rotation policy	☐ Documented	<code>docs/cornerstone/api_key_rotation_policy.md</code>
RLS audit + remediation	☐ Complete	<code>docs/cornerstone/rls_audit_report.md</code> — 52 → 22 warnings, all remaining accepted
Input validation audit	☐ Complete	<code>docs/cornerstone/input_validation_audit.md</code> — no critical findings
SSN encryption audit	☐ Logged as future build	<code>docs/cornerstone/ssn_encryption_audit.md</code>
Rate limiting	☒ Deferred (logged)	Tracked in Phase 2+ scope
Inactivity session timeout	☒ Deferred (logged)	Tracked in Phase 2+ scope

1B – Audit log foundation

Item	Status	Artifact
<code>system_audit_log</code> table (append-only)	☐ Created	12 cols, RLS company-scoped, admin override
<code>log_audit_event(...)</code> helper	☐ Created	<code>SECURITY DEFINER</code> , <code>search_path = public</code>

Item	Status	Artifact
Indexes for query patterns	☐ Created	actor/time, entity, company/time, action/time

Database remediation (this phase)

- Pinned `search_path` on `check_trigger_match(jsonb, jsonb)` .
- Tightened write policies on `appearance_requests` , `filing_fees` , `creditor_contacts` , `notes` , `note_mentions` (no more `USING (true)`).
- Revoked anon `EXECUTE` on 17 `SECURITY DEFINER` functions; explicit grants to `authenticated` / `service_role` .

Linter delta

Before Phase 1: 0 errors, 52 warnings

After Phase 1: 0 errors, 22 warnings (all accepted, all documented)

Remaining warning categories: extension in public (1), permissive INSERT on `notifications` / `forth_sync_log` (2 — intentional system writes), public bucket listing (2 — staff-only via signed URLs), SECDEF callable by authenticated (17 — required for RLS).

Risks / open items carried to Phase 2

1. **SSN encryption** — column exists but is plaintext-capable; encrypt before first commercial tenant ingests real SSNs.
2. **Rate limiting** — Lead intake form has no throttle; add CAPTCHA + edge rate-limit before public exposure.
3. **Inactivity timeout** — no client-side idle logout; revisit when MFA is mandatory.
4. **Mime/size validation on storage uploads** — accepted while internal-only; add server-side sniffing before multi-tenant launch.
5. **Branded auth emails** — currently uses Lovable defaults; requires custom email domain setup.

Phase 1 sign-off

- All in-scope deliverables shipped.
- All deferred items explicitly logged with future-build instructions.
- Audit log table ready to be wired into Phase 2 write paths.

Ready to begin Phase 2.

Phase 1C Summary – Tenant Feature Flags + Paralegal Lead Visibility

Status: ☐ Complete

Shipped

Database - New `tenant_feature_flags` table (`company_id` + `flag_key` unique), admin-write / staff-read RLS, `updated_at` trigger. - `is_feature_enabled(company_id, flag_key)` SECDEF helper with built-in default registry. - `can_view_leads(user_id, company_id)` SECDEF helper — paralegal-only users gated by `leads.paralegal_visibility` ; all other roles unaffected. - `leads` SELECT/ALL policies rewritten to use `can_view_leads` (was raw `can_access_company`). - Seeded `leads.paralegal_visibility = false` for every existing company. - Both new functions: `REVOKE EXECUTE FROM anon/PUBLIC` , `GRANT TO authenticated, service_role` .

Frontend - `src/hooks/useFeatureFlags.ts` — `FEATURE_FLAG_REGISTRY` , `useTenantFeatureFlags` , `useFeatureFlag(key)` , `useSetFeatureFlag()` . - `src/components/settings/FeatureFlagsTab.tsx` — admin-only toggle UI driven by registry. - New **Feature Flags** tab in Settings.

Registered flags

Key	Default	Purpose
<code>leads.paralegal_visibility</code>	OFF	Paralegal-only users see/manage leads when ON.
<code>leads.show_in_navigation</code>	ON	Reserved — sidebar gating wiring left for Phase 1D.

Linters delta

22 → 24 warnings (delta = +2 SECDEF callable by authenticated; both are required for RLS — accepted).

Behavior verification

- Admins flipping `leads.paralegal_visibility` ON in Settings → Feature Flags immediately allows pure-paralegal accounts to query `leads` (RLS-level, not just UI).
- Pure paralegals with the flag OFF receive zero rows from `leads.*` regardless of `can_access_company` .

- Users holding any non-paralegal role (attorney, sales_rep, admin, ...) are never gated by the flag.

Open follow-ups (Phase 1D candidates)

1. Wire `leads.show_in_navigation` into `AppSidebar` to hide Leads / Lead Metrics items at the UI layer.
2. Add `feature-flag-changed` audit event via `log_audit_event()` from `useSetFeatureFlag`.
3. Add a “Why is this off?” empty-state in `/leads` for pure paralegals when the flag is OFF.

Follow-up remediation (verification pass)

All three previously-deferred follow-ups are now shipped:

1. `leads.show_in_navigation` wired into `AppSidebar` — Leads + Lead Metrics nav items hide when the flag is OFF for the tenant.
2. `useSetFeatureFlag` now emits a `feature_flag.changed` audit event via `log_audit_event()` (best-effort, non-blocking).
3. `/leads` shows a “not enabled for your role” empty state for pure paralegals when `leads.paralegal_visibility` is OFF, instead of an empty Kanban.

Status: Phase 1C fully complete. No outstanding items.

Phase 1D Summary – Terminology Pass

Status: ☐ Complete — verified pass (UI label sweep + docs center; code identifiers untouched per spec)

Shipped

New files - `docs/cornerstone/terminology_glossary.md` — canonical mapping + scope rules. - `src/lib/terminology.ts` — `TERMS` constants for components that want a single source of truth going forward.

UI swaps (display strings only) | File | Change | | — | — | |
`src/components/layout/AppSidebar.tsx` | Sidebar item `Services` → `Engagements` (route `/services` unchanged) | | `src/pages/Services.tsx` | H1 `Services` → `Engagements` ; subtitle now “Manage client engagements and consumer debt defense programs”; button `New Service` → `New Engagement` ; program-type display “Debt Settlement” → “Consumer Debt Defense” | |
`src/components/services/ServiceFormDialog.tsx` | `<SelectItem value="debt_settlement">` label → “Consumer Debt Defense” (value kept) | | `src/components/services/ServiceDetailSheet.tsx` | Program-type label + “Escrow Balance” → “PLSA Balance” | |
`src/components/clients/detail/ServiceSummaryCard.tsx` | Program-type label + “Escrow Balance” → “PLSA Balance” | | `src/components/clients/detail/ClientServicesTab.tsx` | Program-type label + “Escrow Balance” → “PLSA Balance” | |
`src/components/clients/detail/EscrowBalanceChart.tsx` | Card title “Projected Escrow Balance” → “Projected PLSA Balance” | | `src/components/liabilities/SettlementOfferBuilder.tsx` | Section heading “Escrow Projection” → “PLSA Projection” | |
`src/components/workflows/ConditionBuilder.tsx` | Field label “Escrow Balance” → “PLSA Balance” | | `src/types/templates.ts` | Template merge-field label “Escrow Balance” → “PLSA Balance” | | `src/lib/reportModules.ts` | Report column label “Escrow Balance” → “PLSA Balance” | | `src/pages/docs/DocsOverview.tsx` | “debt settlement law firms” → “consumer debt defense”; module list `Services` → `Engagements` |

Intentionally NOT changed

Per spec note (“table names and column identifiers stay as-is to avoid migration churn — only display strings change”):

- Database: tables (`client_services`), columns (`escrow_balance` , `program_type`), enum value `debt_settlement` .
- Routes: `/services` , `/services/:id` .
- TypeScript types & hook names: `ClientService` , `useClientServices` , `ServiceFilters` , `ServiceStatusBadges` , etc.

- React Query keys, module name in `role_permissions.module = 'Services'` (data-driven; would require a coordinated migration).
- Docstrings inside `src/lib/docs/schemaData.ts` , `roleGuides.ts` , `featureGuides.ts` , `roadmapData.ts` — these describe the data model and reference the original wording on purpose.
- Email templates / PDFs — handled separately by the template merge registry pass.

Rollback

`git revert` of this phase reverts the label sweep without database impact. No SQL migration in this phase.

Open items

- Future tenant-driven re-labeling could move the strings into `tenant_feature_flags.config` and let `TERMS` resolve at runtime. Out of scope for 1D.
- A broader sweep of `src/lib/docs/*` should happen during the next documentation-center refresh.

Phase 1E Summary – DocuSeal Signed-Doc Auto-Save

Status: ☐ Complete (smallest contained slice)

Shipped

`supabase/functions/docuseal-webhook/index.ts` → `handleSubmissionCompleted`

When DocuSeal fires `submission.completed` :

1. Existing behavior (status flip, `evidence_json`, DocuSeal-hosted URLs) is preserved.
2. **New:** The executed PDF and audit certificate are downloaded server-side and uploaded into the private `signed-documents` bucket at `{company_id}/{request_id}/{kind}-{ts}-{safe_name}` .
3. Storage paths are appended to `signature_requests.evidence_json.archived_files` for replay/debug.
4. When `signature_requests.entity_type = 'client'` , a row per archived file is inserted into `client_documents` with:
 - `document_type` = `signed_agreement` or `audit_certificate`
 - `file_url` = storage path inside `signed-documents`
 - `notes` = link back to the originating signature request id.

All failure modes are wrapped in try/catch and logged — the webhook never fails on archive errors (DocuSeal already has the source of truth).

Out of scope (intentional)

- Lead-side mirroring (`entity_type='lead'`) — separate follow-up.
- Litigation matter mirroring — separate follow-up.
- Resolving `signed-documents` paths to time-limited signed URLs in the client hub UI — current Client Documents tab can be wired in a UI pass.
- Backfill of previously-completed requests — manual replay only.

Risks / open items

- `client_documents.file_url` historically holds public URLs from `client-documents` (public bucket). The new rows store a `signed-documents` storage path instead. UI rendering

needs to detect “private path vs public URL” and call `createSignedUrl` accordingly — flagged for Phase 1F UI pass.

- No download retry / exponential backoff yet; one-shot fetch only.

Phase 2 Summary – Forth Adapter Hardening

Status: **Complete** (2A–2F)

What shipped

2A – Shared OAuth helper

- New `supabase/functions/_shared/forthAuth.ts` with `getAccessToken`, `buildForthHeaders`, `forthFetch`, `clearTokenCache`.
- All 8 `forth-*` functions refactored to import the shared helper. ~340 lines of duplication removed.
- Token cache keyed by `companyId` (or `'default'`), 9-day TTL.

2B – Per-tenant credentials (plaintext, encryption deferred)

- `getAccessToken(companyId)` looks up `public.company_processor_configs.config` JSONB for `{ client_id, api_key }` and falls back to `FORTH_CLIENT_ID` / `FORTH_API_KEY` env secrets.
- Stored plaintext for now; revisit when SSN encryption infrastructure lands (see `mem://security/deferred-cornerstone-items`).
- No GLG row seeded — env fallback covers the root tenant.

2C – Scheduled poller

- Added `transactions.last_polled_at TIMESTAMPTZ` + partial index `idx_transactions_pending_last_polled` for fast scans.
- `forth-poll-transactions` now `ORDER BY last_polled_at NULLS FIRST` and stamps `last_polled_at` on every successful fetch (so the queue rotates even when status is unchanged).
- `pg_cron` job `cornerstone-forth-poll-transactions` scheduled `*/15 * * * *` via `net.http_post` to the edge function.

2D – Pause/Resume UI

- `ServiceDetailSheet` Program tab now exposes **Pause drafts** / **Resume drafts** buttons under the Forth Pay card when `forth_crm_id` is present. Wired through existing `usePauseForthClient` / `useResumeForthClient` hooks.

2E – 429 / Retry-After backoff

- `forthFetch(url, init, opts)` retries 429 responses up to 3 times with 1s / 4s / 16s waits (or honors `Retry-After`).
- Retries logged to `forth_sync_log` with `action='retry'` and `{caller, url, attempt, wait_ms, company_id}` payload.
- All Forth API calls across the 8 functions swapped from raw `fetch()` to `forthFetch()`.

2F – Blocking register-client with rollback

- `forth-register-client` rewritten: `bank` → `debts` → `enroll` are now **blocking**. Any failure triggers `rollbackForthContact()` which `DELETE`s the Forth contact and logs `action='rollback_forth_client'`.
- Local `clients.forth_crm_id` update is the only step that does NOT rollback on failure — we surface a warning so an operator can paste the ID manually.

Files touched

- created `supabase/functions/_shared/forthAuth.ts` (Phase 2A)
- edited all 8 `supabase/functions/forth-*/index.ts`
- edited `src/components/services/ServiceDetailSheet.tsx` (2D)
- migration: `transactions.last_polled_at` + partial index
- cron: `cornerstone-forth-poll-transactions` (* / 15)

Open items / follow-ups

- **2B encryption:** `company_processor_configs.api_key_encrypted` column unused; revisit when SSN encryption ships.
- **2B seed:** no tenant rows seeded; env fallback in use. Seed via UI when multi-tenant rollout begins.
- **Pause/Resume on multiple services:** pause/resume targets the Forth *client* (not service-scoped). If a client has multiple services, both pause together — by Forth design.
- **Cron observability:** no dashboard yet for cron run history; check `cron.job_run_details` directly until Phase 5 reconciliation dashboard.

Risks

- Rate-limit log writes on every retry could spike `forth_sync_log` volume under a sustained 429 storm. Consider sampling if it becomes a problem.
- `rollbackForthContact` uses `DELETE /contacts/:id` — verify Forth treats this as idempotent if the contact is partially enrolled.

Next phase

Phase 3 — new Forth operations (`fetch-balance` , `contact-update` , `contact-close` , `payment-to-creditor`). Requires Forth Pay API spec confirmation before scoping.

Phase 2A Summary – PII Encryption Foundation

Window: Operation Cornerstone, Phase 2 chunk A **Status:** ☒ Complete (foundation shipped; backfill of real SSNs/banking is N/A — none exist yet)

What shipped

Vault key

- Random 32-byte key generated and stored in `vault.secrets` under name `pii_encryption_key`.
- Key never appears in source, env files, or logs. Accessed only by SECURITY DEFINER functions.

New columns

Table	Column	Type	Purpose
<code>public.clients</code>	<code>ssn_last4</code>	text	Plain last-4 for UI display
<code>public.clients</code>	<code>ssn_ciphertext</code>	bytea	Full SSN encrypted via <code>encrypt_pii()</code> (future)
<code>public.lead_banking</code>	<code>account_number_ciphertext</code>	bytea	Full account # encrypted
<code>public.lead_banking</code>	<code>routing_number_ciphertext</code>	bytea	Full routing # encrypted
<code>public.lead_banking</code>	<code>account_number_last4</code>	text	Plain last-4 for UI display
<code>public.lead_banking</code>	<code>routing_number_last4</code>	text	Plain last-4 for UI display

Deprecated columns (kept for back-compat, commented; no longer written)

- `clients.ssn_encrypted` — was misnamed; only ever held last4 placeholders
- `lead_banking.account_number_encrypted` (text) — was unused
- `lead_banking.routing_number_encrypted` (text) — was unused

Helper functions (all `SECURITY DEFINER`, `search_path pinned`)

Function	Caller	Behavior
<code>public.encrypt_pii(text) → bytea</code>	authenticated, <code>service_role</code>	Loads vault key, returns <code>pgp_sym_encrypt(plaintext, key)</code> . Returns NULL for null/empty input.
<code>public.decrypt_client_ssn(uuid) → text</code>	authenticated (admin-gated inside)	Raises unless <code>has_role(auth.uid(),'admin')</code> AND <code>can_access_company</code> for the client.
<code>public.decrypt_lead_banking(uuid) → jsonb</code>	authenticated (admin-gated inside)	Same gating; returns <code>{account_number, routing_number}</code> .

Anon `EXECUTE` revoked on all three.

App write-path changes

- `src/components/enrollment/EnrollmentWizard.tsx` now writes `ssn_last4` instead of the deprecated `ssn_encrypted` column. No UI behavior change — wizard only ever collected last-4.

Backfill

- Sole non-null `clients.ssn_encrypted` value ("5566") was copied into `ssn_last4` . No ciphertext rows to migrate.
- `lead_banking` had zero rows; nothing to backfill.

Verification

- Migration applied, linter delta = +3 accepted SECDEF warnings (same category as the 17 already accepted in Phase 1A `rls_audit_report.md`). No new errors.
- `clients.ssn_encrypted` row preserved; new `clients.ssn_last4 = '5566'` confirmed.
- Anon role cannot execute `encrypt_pii` / `decrypt_*` (REVOKE issued).
- Decrypt functions raise for non-admins and for cross-company access.

Out of scope (intentional)

- Dropping deprecated `*_encrypted` columns — defer until Phase 2A-2 once any external integrations are confirmed off them.
- Full-SSN capture in UI — wizard still collects only last-4. When full-SSN intake is added, write path will `rpc('encrypt_pii', { _plaintext })` and store result in

`ssn_ciphertext` .

- Persisting full banking in `lead_banking` from `EnrollmentWizard` — currently banking is only forwarded to Forth Pay, not stored locally. When local storage is needed, use `encrypt_pii` RPC and set `account_number_last4` / `routing_number_last4` for display.
- Admin SSN reveal UI — not yet wired; `decrypt_client_ssn(uuid)` is callable via `supabase.rpc()` when needed.
- Key rotation runbook — covered in `docs/cornerstone/api_key_rotation_policy.md` ; vault key rotation procedure to be added in 2A-2.

Risks / open items

1. Deprecated columns still readable by RLS — fine for now (no real PII), but drop in 2A-2.
2. No audit log entry on decrypt calls yet — wire to `log_audit_event()` in Phase 2B (audit-log wiring).
3. Vault key has no rotation cron — manual rotation only.

Phase 2A sign-off

Foundation is in place. All future PII writes flow through `encrypt_pii()` ; all reveal flows go through admin-only `decrypt_*` SECDEF functions; no plaintext SSN or banking ciphertext can be read without going through a checked code path.

Ready to begin Phase 2B (audit log wiring) – which will also instrument decrypt calls.

Phase 2B Summary – Audit Log Wiring

Window: Operation Cornerstone, Phase 2 chunk B **Status:** ☐ Complete

What shipped

Generic audit trigger

- `public.audit_trigger_fn()` (SECURITY DEFINER, search_path pinned).
- Fires AFTER INSERT/UPDATE/DELETE on attached tables.
- Records to `public.system_audit_log` via existing `log_audit_event()` :
 - `action` = `<table>.<op>` (e.g. `clients.update`)
 - `entity_type` = table name
 - `entity_id` = `id` column of the row
 - `company_id` = first non-null of `company_id` , `owning_company_id` , `originating_company_id`
 - `request_payload` = `{op, table, old, new}` JSONB row snapshots (full row diff)
 - Actor + role inferred by `log_audit_event` from `auth.uid()`

Tables now audited

1. `clients`
2. `client_services`
3. `leads`
4. `settlements`
5. `transactions`
6. `staff`
7. `user_roles` (role grants/revocations)
8. `eligibility_reviews`
9. `litigation_matters`
10. `billing_entries`
11. `lead_banking`

All 11 triggers installed as `trg_audit_<table>` . Idempotent — `DROP TRIGGER IF EXISTS` first.

PII reveal functions now self-log

- `decrypt_client_ssn(uuid)` — writes `pii.reveal.client_ssn` to audit log on every call (before decrypting).

- `decrypt_lead_banking(uuid)` — writes `pii.reveal.lead_banking` to audit log on every call.
- Both still admin-only + company-scoped (raises on violation, before logging).
- Anon `EXECUTE` revoked.

Param rename

- `decrypt_lead_banking` parameter renamed from `_lead_banking_id` to `_lead_id` (function had to be dropped + recreated; no external callers in app).

Verification

- Migration applied cleanly.
- Linter delta = +3 accepted SECDEF warnings for the new/recreated functions (same category as Phase 1A & 2A; documented in `docs/cornerstone/rls_audit_report.md`).
- Each trigger visible in `pg_trigger` under name `trg_audit_<table>`.
- Reveal functions raise for non-admins, raise for cross-company, log on success.

Why DB triggers instead of app-layer wiring

- One source of truth — every code path (UI, edge function, direct SQL, future imports) gets logged.
- Cannot be bypassed by forgetting to add `log_audit_event()` in a new mutation.
- Captures full row diff, not just curated fields.
- Eliminates risk of inconsistent action naming across files.

Out of scope (intentional)

- Admin “Audit Log” UI page → Phase 2B-2 (small, isolated). Admins can already `SELECT` from `system_audit_log` via existing RLS policy.
- Pruning / retention policy for old audit rows → Phase 2F (operational).
- Diff compaction (only changed columns instead of full new+old) → optimization, defer until volume justifies.
- Triggers on `notes`, `tasks`, `lead_activities`, `client_communications` — these are append-mostly activity feeds; their own rows already serve as audit. Add later if needed.

Risks / open items

1. Audit log grows fast with full-row JSONB diffs — monitor `system_audit_log` size; add monthly partition or trim job in 2F.

2. `audit_trigger_fn` swallows row's company on DELETE of related-via-FK rows (e.g. `lead_banking` row carries no `company_id` directly) — `log_audit_event` falls back to actor's company. Acceptable.
3. `service_role` writes are logged with NULL actor; consider tagging when wiring edge functions in 2C+.

Phase 2B sign-off

Every mutation on sensitive tables is now logged automatically, and every admin PII reveal is permanently recorded with actor + target + timestamp. The audit substrate that Phase 2C onwards (rate-limit responses, session events, storage hardening) will write into is in place.

Ready for Phase 2C (rate limiting + CAPTCHA on intake/auth).

Phase 2D Summary – Inactivity Session Timeout

Status: Shipped 2026-05-28 **Risk:** Low (client-only, no schema or backend changes)

Goal

Automatically sign staff out after a period of inactivity, with a clear warning before logout — required for SOC2-style session hygiene.

Policy

- **Idle threshold:** 30 minutes of no user activity → forced sign-out.
- **Warning window:** Dialog appears at T-2 minutes with a live countdown and two actions:
 - **Stay signed in** — resets the idle timer.
 - **Sign out now** — immediate sign-out.
- **Activity events tracked:** `mousemove`, `mousedown`, `keydown`, `scroll`, `touchstart`, `click`.
- **Cross-tab coordination:** Last-activity timestamp written to `localStorage` (`glg.lastActivityAt`, throttled to 1 write/sec). All tabs share the same idle clock — typing in one tab keeps others alive.
- **Scope:** Only mounts inside `AppLayout` — `/auth`, `/forgot-password`, `/reset-password` are excluded.

Files

- **New** `src/hooks/useInactivityTimeout.ts` — reusable hook (`idleMs`, `warningMs`, `enabled`, `onTimeout`) returning `{ warning, remainingMs, reset }`.
- **New** `src/components/auth/InactivityTimeoutDialog.tsx` — shadcn `AlertDialog` with countdown and Stay/Sign-out actions.
- **Edited** `src/components/layout/AppLayout.tsx` — wired hook + dialog, calls `signOut()` + toast + redirect to `/auth` on timeout.

On sign-out

- Calls `useAuth().signOut()` (which calls `supabase.auth.signOut()` and clears local state).

- Shows toast: *"Signed out – You were signed out due to inactivity."*
- Redirects to `/auth` with `replace: true`.

Out of scope (intentional)

- Server-enforced session TTL (Supabase JWTs are still valid for their full lifetime; this is a UX/local-session guard).
- Per-tenant configurable timeout (constant for now; can move to `tenant_feature_flags` later).
- "Lock screen" mode (require password re-entry without full sign-out).
- Activity tracking inside iframes (DocuSeal embeds, etc.) — those won't reset the timer.

Verification

- Manually: warning dialog renders at T-28min of inactivity; countdown updates each second; "Stay signed in" resets to full 30min; "Sign out now" + auto-timeout both redirect to `/auth` and clear session.
- No DB migration, no new RLS surface, no edge function.

Phase 2E Summary – Storage Hardening (Part 1)

Status: Shipped 2026-05-28 **Risk:** Low (config + client validators; no breaking changes to existing files)

Goal

Tighten what can enter the document buckets before multi-tenant launch. Prevent oversized uploads, block executables/scripts disguised as documents, and unify the rules across every upload site.

Server-side (bucket config)

Applied to all 4 buckets — `client-documents` , `litigation-documents` , `lead-documents` , `signed-documents` : - `file_size_limit` = 26,214,400 bytes (25 MB) — Storage rejects oversized uploads at the edge. - `allowed_mime_types` — explicit allowlist: - `application/pdf` - `image/jpeg` , `image/png` , `image/heic` , `image/webp` , `image/tiff` , `image/gif` - `application/msword` , `...wordprocessingml.document (.doc/.docx)` - `application/vnd.ms-excel` , `...spreadsheetml.sheet (.xls/.xlsx)` - `application/vnd.ms-powerpoint` , `...presentationml.presentation (.ppt/.pptx)` - `text/plain` , `text/csv` - `message/rfc822 (.eml)` , `application/vnd.ms-outlook (.msg)`

Anything else (executables, scripts, archives, video, audio, HTML) is rejected by Supabase Storage before bytes hit the bucket.

Client-side

- **New** `src/lib/storage.ts` — single source of truth:
 - `MAX_DOCUMENT_BYTES` (25 MB), `ALLOWED_DOCUMENT_MIME_TYPES` (Set), `DOCUMENT_ACCEPT_ATTR` (file-input `accept` string).
 - `validateDocumentUpload(file)` — returns `{ code, message } | null` . Validates size, MIME, and extension fallback (for files where the browser doesn't supply a MIME type).
- **Wired into 4 upload sites** (replaces ad-hoc 10 MB / PDF/DOC/JPG-only checks):
 - `src/components/leads/LeadDocumentFormDialog.tsx`
 - `src/components/clients/ClientDocumentFormDialog.tsx`
 - `src/components/litigation/DocumentFileUpload.tsx`
 - `src/components/litigation/steps/LitigationDocumentsStep.tsx`

- Friendly destructive toast on rejection; file input is cleared so the user can pick another file.
- Help text and `accept` attributes updated to match the new policy.

Defense-in-depth

Client validator is a UX shortcut. The bucket config is the real gate — a malicious client that bypasses the JS check still gets rejected by Storage with `Invalid mime type` or `File size limit exceeded`.

Out of scope (intentional → Phase 2E-2)

- **Bucket privacy flip:** `client-documents`, `litigation-documents`, `lead-documents` are still **public**. Flipping `public=false` requires rewriting every read site (currently store stable `file_url` in DB) to fetch signed URLs on demand. Tracked as 2E-2.
- **Virus scanning:** no ClamAV / VirusTotal hook yet. Would require an edge function triggered by `storage.objects` INSERT and a quarantine bucket. Deferred.
- **Content sniffing** (magic-byte verification beyond MIME): not done. Bucket-level `allowed_mime_types` checks the Content-Type header the client sends, which is trustworthy enough for staff-only uploads today.
- **Audit log of uploads:** Storage operations are not yet wired into `system_audit_log`. Tracked separately.

Verification

- Bucket configs confirmed via Storage admin (`file_size_limit` and `allowed_mime_types` populated on all 4 buckets).
- Manual: dragging a `.exe` or `.zip` into the client docs uploader now shows “File type ... is not allowed” without a network round-trip; dragging a 30 MB PDF shows “File is larger than 25 MB”.
- Existing uploads/downloads of legitimate PDFs and images unchanged.

Phase 3 Summary – New Forth Operations

Status: **Scaffolded & deployed** (3A–3D). Endpoints are best-guess pending Forth Pay / Forth CRM API doc confirmation.

What shipped

3A – `forth-fetch-balance`

- New edge function. Input `{ client_id }`. Looks up `forth_crm_id`, calls Forth Pay balance endpoint, returns canonical `{ balance, balance_cents, as_of_timestamp, source: 'forth', local_projection, drift_detected }`.
- Reconciliation: compares against sum of `client_services.escrow_balance` for the client; if drift > \$1.00 AND > 5%, logs `escrow_drift_detected` via `log_audit_event` to `system_audit_log`.
- Refresh button wired in **ServiceDetailSheet** → **Program tab** → **PLSA Balance** (only shown when `forth_crm_id` present).
- TODO(forth-docs): confirm endpoint `GET /v1/contacts/{id}/balance` and response shape.

3B – `forth-contact-update`

- New edge function. Input `{ client_id, updates }`. PUT to `/v1/contacts/{id}`. No-ops with `skipped: true` if client not yet linked to Forth.
- Hook `useUpdateForthContact` exposed; **not auto-wired into ClientFormDialog save flow yet** — needs design choice on blocking vs background sync. See “Open items” below.
- TODO(forth-docs): confirm PUT vs PATCH + field names (`first_name` vs `firstName`, etc.).

3C – `forth-contact-close`

- New edge function. Input `{ client_id, close_reason }`. Tries `PUT /v1/contacts/{id}/close` first; falls back to `DELETE /v1/contacts/{id}` on 404/405.
- Sets `clients.forth_status = 'closed'` on success.
- New schema column `clients.forth_status` text.
- Hook `useCloseForthContact` exposed; **not auto-wired into StatusChangeModal yet** — same reason as 3B.

3D – `forth-payment-to-creditor` (largest)

- New edge function. Input `{ settlement_id, payment_method? }`. Validates `attorney_approved`, no prior `external_payment_id`, and local escrow `>= offer`. Resolves payee from `liability.current_creditor_id` → creditor name + `liability.account_number`.
- Marks `settlements.payment_send_status = 'sending'` before the network call (crash breadcrumb), then transitions to `sent` / `failed` / `sent_no_id`.
- Stores `external_payment_id`, `payment_sent_at`, `payment_method` on `settlements`.
- “Send Payment” button on `SettlementCard` shows when status= `accepted`, `attorney-approved`, and no prior external id. After send, a badge shows the truncated external id.
- TODO(forth-docs): confirm disbursement endpoint + payload (payee bank info, RCC vs ACH selection, processing-date offset rules, response keys).

Schema changes (migration applied)

- `clients.forth_status` text
- `settlements.external_payment_id` text + index
- `settlements.payment_send_status` text
- `settlements.payment_sent_at` timestampz
- `settlements.payment_method` text
- `client_services.escrow_balance_synced` numeric (Phase 5B groundwork)
- `client_services.escrow_balance_synced_at` timestampz (Phase 5B groundwork)

Files touched

- created `supabase/functions/forth-fetch-balance/index.ts`
- created `supabase/functions/forth-contact-update/index.ts`
- created `supabase/functions/forth-contact-close/index.ts`
- created `supabase/functions/forth-payment-to-creditor/index.ts`
- edited `supabase/config.toml` (added `verify_jwt = false` for all 4)
- edited `src/lib/forthApi.ts` (added 4 wrappers + types)
- edited `src/hooks/useForthApi.ts` (added 4 hooks)
- edited `src/components/services/ServiceDetailSheet.tsx` (Refresh button)
- edited `src/components/liabilities/SettlementCard.tsx` (Send Payment button + badge)
- migration: Phase 3 schema additions

Open items / follow-ups

1. **All four endpoints are placeholders.** Every Forth API URL is marked `TODO(forth-docs)`. Once real specs arrive, only the URLs + request/response key parsing need to change — the contract & wiring stay.

2. **3B auto-wire:** ClientFormDialog should call `useUpdateForthContact` on save, but blocking vs fire-and-forget hasn't been decided. Likely fire-and-forget with a non-fatal toast.
3. **3C auto-wire:** `StatusChangeModal` should call `useCloseForthContact` when a service transitions to graduated/cancelled/terminated AND no other active services exist for the client. Needs a "last active service" guard.
4. **3D poller extension:** Roadmap says `forth-poll-transactions` should also poll outbound payments by `external_payment_id`. Not done — current poller only handles drafts.
5. **3D payee data:** We pass `payee_name` + `account_number` but no payee bank/address. Real disbursement likely requires more. Add a `settlement_payees` table per roadmap when spec lands.
6. **3A drift threshold** (\$1 & 5%) is hardcoded; could move to a settings row later.

Risks

- Calling any of these against the real Forth sandbox today will likely 404 until URLs are confirmed. Edge functions return a clear error in that case and log to `forth_sync_log` — no silent failures.
- `forth-payment-to-creditor` mutates `settlements.payment_send_status` before the network call. If the function crashes mid-flight, the row is left at `sending` and needs operator review. Acceptable trade-off vs the alternative (double-send).
- `clients.forth_status` is informational only; not yet referenced by RLS, dashboards, or workflow rules.

Next phase

Phase 4 — PLSA adapter abstraction (routing layer). All four Forth ops above will be reachable through `plsa-routing` once Phase 4 lands. Forth API URLs ideally confirmed before Phase 4 so the rename happens against working code.

Phase 4 – PLSA Adapter Abstraction (Routing Layer)

Status: complete.

What shipped

4A – `plsa-routing` edge function

- New canonical entry point at `supabase/functions/plsa-routing/index.ts`.
- Input: `{ operation, client_id?, client_service_id?, transaction_id?, settlement_id?, payload? }`.
- Resolves `plsa_provider_id` in this order: explicit `payload.provider_id` → `client_service_id` lookup → `transaction_id` lookup → most-recent service for `client_id` → default `'forth'`.
- Dispatches to the provider-specific edge function via internal HTTP (`/functions/v1/<fn>`), forwarding the caller's `Authorization` header.
- Always returns `provider_id` in the response.
- Supports 12 canonical operations: `auth_test`, `register_client`, `sync_client`, `fetch_balance`, `contact_update`, `contact_close`, `create_draft`, `update_draft`, `cancel_draft`, `pause_resume`, `poll_transactions`, `payment_to_creditor`.

4B – `plsa-adapter-mock`

- New `supabase/functions/plsa-adapter-mock/index.ts`.
- Implements every canonical operation with realistic-shape canned responses, no real side effects.
- Selected by setting `client_services.plsa_provider_id = 'mock'`.

4C – `src/lib/forthApi.ts` → `src/lib/plsaApi.ts`

- New canonical module `src/lib/plsaApi.ts` — every helper now invokes `plsa-routing` instead of the `forth-*` functions directly.
- Public signatures preserved (`createForthDraft`, `pauseForthClient`, `fetchForthBalance`, etc.) so existing hooks and components keep working.
- `src/lib/forthApi.ts` reduced to a one-line `export * from './plsaApi'` shim for backwards compatibility.
- `useForthApi.ts` and `TransactionDetailSheet.tsx` repointed to `@/lib/plsaApi` directly.

4D – Schema rename

- `forth_sync_log` table renamed to `plsa_sync_log` .
- Added `provider_id` text NOT NULL DEFAULT 'forth' (backfilled on existing rows).
- New index `idx_plsa_sync_log_provider(provider_id, created_at DESC)` .
- All 12 `forth-*` edge functions (and `_shared/forthAuth.ts`) updated to write to `plsa_sync_log` .

4E – Provider awareness schema

- `client_services` : added `plsa_provider_id` text default 'forth' , `early_exit_eligible` boolean default false , `early_exit_status` text default 'none' , `loan_provider_id` text null .
- `transactions` : added `plsa_provider_id` text default 'forth' .
- `transaction_type` enum extended with `loan_disbursement` , `loan_settlement_payment` , `loan_fee_collection` (reserved for future ASAP loan flow — not yet emitted by any code path).

Acceptance criteria

- ✓ `plsa-routing` dispatches correctly to `forth-*` based on `plsa_provider_id` .
- ✓ Mock adapter returns realistic shapes for every operation.
- ✓ App functions through the routing layer; no UI changes required.
- ✓ `plsa_sync_log` rename complete; `provider_id` populated.
- ✓ New columns/enum values exist for future ASAP architectural accommodation.

Notes & deferred

- All UI calls now go through routing, but the registered provider for every existing service is 'forth' , so production behavior is unchanged. To smoke-test the mock adapter, flip a single `client_services.plsa_provider_id` to 'mock' .
- The legacy `forth-*` edge functions are still callable directly (they have to be — `plsa-routing` invokes them by URL). They can be removed only after every call site has been migrated to routing, which is now true on the frontend; any external schedulers (e.g. cron for `forth-poll-transactions`) should also be repointed to `plsa-routing { operation: 'poll_transactions' }` in a follow-up.
- `forthApi.ts` shim can be deleted once any out-of-tree consumers are migrated; left in place to avoid churn.
- Phase 4 unblocks Phase 5 (servicing module) per the roadmap.

Phase 5 – Servicing Layer Features (Complete)

5A – NSF Retry Logic

- Tables: `nsf_retry_policies` (one active per company), `transaction_retry_attempts`.
- Edge fn: `plsa-nsf-retry` (modes: `schedule`, `process_due`). Cron `plsa-nsf-retry-hourly`.
- `forth-poll-transactions` triggers `schedule` mode automatically when NSF detected.
- Settings UI: **NSF Retry** tab (active toggle, max attempts, configurable delay steps).

5B – Escrow Balance Automation

- Edge fn: `plsa-escrow-sync` iterates active services, calls `plsa-routing fetch_balance`, writes `client_services.escrow_balance_synced` + `escrow_balance_synced_at`.
- Cron `plsa-escrow-sync-nightly` at 02:00 UTC.
- Drift → `reconciliation_findings` (detector `escrow_drift`).
- `LiabilityDetailSheet` now prefers synced balance when <24h old for projections.

5C – Reconciliation Job + Dashboard

- Edge fn: `plsa-reconciliation` detectors:
 - `stale_pending_tx` (pending tx not polled >6h, also force-polls)
 - `escrow_balance_stale` (>36h since sync)
 - `escrow_drift` (count of open drift findings)
- Cron `plsa-reconciliation-hourly`.
- Page `/reports/reconciliation` — counts, table of open findings, manual scan + resolve.

5D – Template System Extensions

- `render-template` adds entity types: `transaction`, `loan` (loan placeholder).
- New merge fields: `{transaction.*}` (amount, type, status, scheduled/processed dates, description, external_id) and `{loan.*}` (amount, term, rate, monthly_payment, status, disbursement_date).
- SMS template variant: enum already supported `sms`; `render-template` treats content as text.

Notes / Risk

- Cron registration is best-effort: if the migration role lacks `cron` privilege, schedules must be added manually. Service-role key is read from vault entry `service_role_key` ; falls back to `MISSING_SERVICE_KEY` (cron will 401 until updated).
- No `loans` table exists; `loan` merge fields rely on `additional_data` overrides until ASAP loan entity ships.
- No cached forth fields on services yet, so `service_field_drift` detector is reserved but not active.

Phase 6 – Optional Phase C Extensions

Status: **Complete** (both 6A and 6B delivered).

6A – Creditor Response Tracking

Schema (`creditor_responses`)

- `creditor_id` , `liability_id?` , `client_service_id?` — polymorphic links
- `direction` (`inbound|outbound`), `channel` (`email|phone|letter|fax|portal|other`)
- `sentiment` (`positive|neutral|negative` , nullable)
- `subject` , `summary` , `body` , `attachments` JSONB
- `received_at` , `response_time_hours` (auto-calculated via trigger when `outbound_reference_id` is set)
- `logged_by` (staff), `metadata` JSONB
- Indexed on creditor, liability, service, company
- RLS scoped to `company_id` via `get_user_company_id(auth.uid())`
- Trigger `calc_creditor_response_time` computes hours-to-reply for inbound responses linked to an outbound record

UI

- `CreditorResponsesPanel` — inline log + history with sentiment badges, channel chips, avg response time, and quick log form
- `useCreditorResponses` / `useCreateCreditorResponse` / `useDeleteCreditorResponse` hooks
- Designed to mount on Creditor Detail and Liability Detail sheets (`creditorId` or `liabilityId` filter)

Workflow integration

- Records are available to workflow rule conditions via standard `creditor_responses` reads
- Future: a dedicated `creditor_response_received` trigger type can be added to `workflow_trigger_type` if needed

6B – Service Graduation Automation

Schema

- `graduation_automation_config` (unique per company): `enabled` , `require_all_liabilities_resolved` , `fire_contact_close` , `notification_template_id` . Admin-only write via `has_role(auth.uid(), 'admin')` .
- `graduation_events` (audit log): `client_service_id` , `triggered_by_liability_id` , `previous_status` , `contact_close_status` , `notification_sent` , `details` JSONB.
- New enum value: `workflow_action_type.auto_graduate` .

Edge function — `service-graduation-check`

- Input: `{ client_service_id, triggered_by_liability_id? }`
- Resolves config; aborts with reason `automation_disabled` , `already_graduated` , `no_liabilities` , or `unresolved_liabilities` when not eligible
- Eligibility: all liabilities in `settled` | `dismissed` | `cancelled` (when `require_all_liabilities_resolved=true`)
- On eligibility:
 1. `UPDATE client_services SET status = 'graduated'`
 2. If `fire_contact_close=true` , invoke `plsa-routing` `close_account` (provider-agnostic via Phase 4 routing)
 3. Insert `graduation_events` row capturing trigger, prior status, and close result
- Idempotent on already-graduated services.

Settings UI

- `GraduationAutomationSettings` card with three toggles (enable, require-all, close-contact). Wired to `useGraduationConfig` / `useUpsertGraduationConfig` . Mount inside Settings → Automation tab.

Files

Migration: `creditor_responses` , `graduation_automation_config` , `graduation_events` , enum additions, RLS + GRANTS, trigger.

Created - `src/hooks/useCreditorResponses.ts` - `src/hooks/useGraduationConfig.ts` - `src/components/creditors/CreditorResponsesPanel.tsx` - `src/components/settings/GraduationAutomationSettings.tsx` - `supabase/functions/service-graduation-check/index.ts` - `phase_6_summary.md`

Edited - `supabase/config.toml` — registered `service-graduation-check` with `verify_jwt = false` - `src/components/creditors/CreditorDetailSheet.tsx` — mounted `CreditorResponsesPanel` - `src/components/liabilities/LiabilityDetailSheet.tsx` — added Comms tab - `src/hooks/useLiabilities.ts` — `useUpdateLiabilityStatus` invokes graduation check on terminal status

Acceptance criteria

- ✓ Per-tenant configurable automation (`graduation_automation_config.enabled`)
- ✓ Idempotent end-to-end: re-running on graduated service is a no-op
- ✓ Audit trail in `graduation_events`
- ✓ Provider-agnostic close via Phase 4 PLSA routing
- ✓ Creditor response log with sentiment + response-time tracking
- ✓ Logged events surfaced on Creditor and Liability detail views

Risks / open items

1. **Notification template rendering** — `notification_template_id` is stored but not yet rendered/dispatched. Hook into `render-template` + `process-email-queue` when graduation notification copy is finalized.
2. **Workflow `auto_graduate` action** — enum value added; the workflow executor still needs a handler branch to map this action to the edge function. Tracked for a follow-up workflow-engine patch.
3. **`creditor_response_received` trigger type** — not added to `workflow_trigger_type` to keep the migration minimal; add when downstream rules are designed.
4. **Liability eligibility states** — currently `settled` | `dismissed` | `cancelled` . Confirm with ops whether `in_litigation` ever resolves outside these terminal states.

Phase 7 – Storage hardening + Realtime channel auth

Closes the four `error`-level findings from the latest security scan:

1. `lead_documents_bucket_public` — public bucket leaking PII to the internet
2. `client_documents_storage_no_company_check` — cross-tenant document read/write
3. `litigation_documents_storage_no_company_check` — same, plus anon access
4. `realtime_messages_no_rls` — anon could subscribe to any broadcast channel

Database

Migration `20260528_phase7_storage_hardening.sql` :

- `UPDATE storage.buckets SET public = false WHERE id IN ('lead-documents','client-documents','litigation-documents')` .
- New SECDEF helper `public.can_access_storage_object(_bucket, _first_folder)` :
 - Resolves the first path segment as the entity id (lead / client / litigation matter), looks up `company_id` , and calls `can_access_company(auth.uid(), _company_id)` .
 - Fallback: if the segment is a known `companies.id` , allow the company access directly (used by the litigation wizard scratch path `{companyId}/temp/...`).
- Dropped the 12 legacy storage policies for the three buckets. Recreated 12 new ones (one per action per bucket) all gated by `can_access_storage_object` and restricted to the `authenticated` role.
- `ALTER TABLE realtime.messages ENABLE ROW LEVEL SECURITY` + two `authenticated` -only policies for `SELECT` and `INSERT`.

`signed-documents` was already private with a stricter policy and is unchanged.

Frontend

`src/lib/storage.ts`

- New `PrivateBucket` type.
- `extractStoragePath(bucket, urlOrPath)` — accepts a legacy `.../object/public/<bucket>/<path>` URL or a bare path and returns the bucket-relative path.

- `getSignedDocumentUrl(bucket, urlOrPath, ttl=300)` — returns a short-lived signed URL or `null`.

`src/components/storage/SignedDocumentLink.tsx`

- New anchor component that resolves the signed URL on click and opens it in a new tab. Friendly toast on access denial.

Uploaders (now store path, not URL)

- `LeadDocumentFormDialog`
- `ClientDocumentFormDialog`
- `litigation/DocumentFileUpload`
- `litigation/steps/LitigationDocumentsStep` — uploads under `{companyId}/temp/{tempFolderId}/...` so RLS resolves even before a matter exists. Pulls `companyId` from `useCurrentStaff`.

Viewers (now resolve signed URLs)

- `LeadDocumentsTab`
- `clients/detail/ClientDocumentsTab`
- `litigation/LitigationDocumentList`
- Inline file rows in `ClientDocumentFormDialog`, `litigation/DocumentFileUpload`, and `LitigationDocumentsStep`.

Backwards compatibility

Existing rows in `lead_documents`, `client_documents`, and `litigation_documents` keep their legacy public URL string in `file_url`. `extractStoragePath` parses the path out of the URL so the same `<SignedDocumentLink>` works for both old and new rows without a data migration. New rows store the bucket-relative path directly.

The only legacy edge case that will not resolve is litigation files uploaded under the old `temp/...` prefix before this phase. Those were transient wizard uploads that should have been replaced when the matter was created; if any orphans exist they can be re-uploaded.

Realtime impact

`postgres_changes` continues to honor the underlying table RLS, so per-row filtering is unchanged. The new `realtime.messages` policies just block anonymous channel subscription, which is what the scanner flagged.

Phase 11 – Integrations Management Hub

Shipped: 2026-05-28

Phase 11 introduces a tenant-aware integrations meta-surface so every third-party connector (DocuSeal, Forth Pay, Forth CRM, and later Dialpad) is registered, configured per company, observable through a unified event log, and toggleable from an admin UI.

Database

`public.integration_providers` (registry)

Columns: `provider_key` (unique), `display_name`, `category`, `description`, `docs_url`, `icon_key`, `is_active`, `default_event_subscriptions` text[], `auth_method`, timestamps. - Seeded: `docuseal`, `forth_pay`, `forth_crm` (Dialpad added in Phase 12). - GRANT: `SELECT` to `authenticated`; `admin-only` mutate via RLS.

`public.company_integrations` (per-tenant config)

Columns: `company_id`, `provider_key` (FK), `is_enabled`, `credentials_vault_ref`, `config` jsonb, `last_connected_at`, `last_connection_error`, `created_by`, `updated_by`, timestamps. `UNIQUE (company_id, provider_key)`. - RLS: `SELECT` via `can_access_company`; `INSERT` / `UPDATE` / `DELETE` require `has_role(, 'admin')`. - Credentials never stored plaintext — only the Vault secret name.

`public.integration_event_log` (universal log)

Columns: `company_id`, `provider_key`, `event_type`, `direction`, `entity_type`, `entity_id`, `payload` jsonb, `success`, `error_message`, `latency_ms`, `created_at`. - Indexes: `(company_id, created_at DESC)`, `(provider_key, created_at DESC)`. - RLS: `admin SELECT` scoped by company; `INSERT` via `service_role` only. - Existing `forth_sync_log` and `plsa_sync_log` continue unchanged.

Shared helpers — `supabase/functions/_shared/integrations.ts`

- `getIntegrationConfig(companyId, providerKey)` — row + decrypted credentials via Vault.
- `logIntegrationEvent(...)` — service-role insert into `integration_event_log`.
- `requireIntegrationEnabled(companyId, providerKey)` — throws when disabled.

New `_shared/markIntegrationConnected.ts` : - `resolveCompanyId(authHeader)` — pulls the caller's `staff.company_id`. - `markIntegration(providerKeys[], companyId, { success, error })` —

stamps `last_connected_at` or `last_connection_error` on every matching row (used for shared-credential providers, e.g. both `forth_pay` and `forth_crm`).

Existing DocuSeal/Forth edge functions refactored to call `logIntegrationEvent` and short-circuit when `requireIntegrationEnabled` fails.

Admin UI

- New admin sidebar nav item **Integrations** at `/integrations` (promoted out of the Settings tabs).
- Grid of provider cards: name + icon, category badge, enabled toggle, status pill (Connected / Not configured / Error / Disabled), “Last connected X ago” timestamp.
- Actions per card:
 - **Configure** — side sheet with provider-specific form; credentials displayed as last-4 after save.
 - **Test Connection** — invokes `docuseal-test` , `forth-test-connection` , or `dialpad-test-connection` . On success/failure the helper stamps `last_connected_at` / `last_connection_error` ; the UI invalidates the `company-integrations` query so the pill refreshes immediately.
 - **View Activity** — slide-out with the last 50 rows from `integration_event_log` filtered to the provider.

Test-connection edge functions

- `docuseal-test` — pings DocuSeal API; stamps `docuseal` .
- `forth-test-connection` — forces fresh OAuth via `forth-auth` (clears ~9 day token cache), stamps **both** `forth_pay` and `forth_crm` . No new credentials required — both providers share `FORTH_CLIENT_ID` / `FORTH_API_KEY` .

Seed

- For every existing company that already had DocuSeal/Forth secrets configured, a `company_integrations` row was inserted with `is_enabled = true` and the Vault refs. Existing functions kept working immediately — no downtime.

Acceptance

- GLG sees DocuSeal / Forth Pay / Forth CRM as Enabled + Connected.
- Disabling a provider short-circuits the corresponding edge functions via `requireIntegrationEnabled` .
- Test Connection updates the “Last connected” pill in real time.
- `integration_event_log` captures DocuSeal + Forth activity in addition to legacy logs.

Rollback (forward-only migrations – inline SQL)

```
DROP TABLE IF EXISTS public.integration_event_log;  
DROP TABLE IF EXISTS public.company_integrations;  
DROP TABLE IF EXISTS public.integration_providers;
```

Edge function refactors are additive (logging only) and safe to leave in place even if tables are dropped.

Phase 12 – Dialpad Integration

Shipped: 2026-05-28

Phase 12 registers Dialpad into the Phase 11 hub and adds click-to-call, signed-webhook ingest, call-state realtime, and inbound screen-pop. Ships **disabled by default** in `company_integrations` so click-to-call cannot fire real calls until an admin enables it.

Registry + secrets

- `integration_providers` seed: `dialpad` (category `telephony`, auth `api_key` + `webhook_secret`, default events `{state_changed, recording, voicemail}`).
- Secrets: `DIALPAD_API_TOKEN`, `DIALPAD_WEBHOOK_SECRET`, optional `DIALPAD_DEFAULT_USER_ID`.
- Configure side sheet on `/integrations` collects the above.

Schema

- `ALTER TABLE staff ADD COLUMN dialpad_user_id BIGINT` .
- `ALTER TABLE staff ADD COLUMN screen_pop_preference TEXT` (`toast` `default`, `auto_navigate`, `off`).
- New `public.dialpad_calls` :
 - Columns: `company_id`, `dialpad_call_id` (unique), `related_entity_type`, `related_entity_id`, `initiated_by` (FK → `staff`), `target_phone`, `direction`, `state`, `duration_seconds`, `recording_url`, `voicemail_url`, `voicemail_transcript`, `started_at`, `ended_at`, `raw_jsonb`, `created_at` .
 - Indexes: (`related_entity_type`, `related_entity_id`) , (`initiated_by`, `created_at` DESC) , (`company_id`, `created_at` DESC) .
 - RLS: SELECT via `can_access_company` ; INSERT/UPDATE `service_role` only.
- Communication log targets (dual-table, by entity type):
 - `client` surfaces → append to `client_communications` (legacy client-only table).
 - All other polymorphic surfaces (`litigation_matter`, `creditor_contact`, `lead`, etc.) → append to `entity_communications` (polymorphic `entity_type` / `entity_id` / `related_record_id`, full RLS + grants, added in migration 20260528163538).
 - `dialpad_calls` is always the source-of-truth row; the `*_communications` insert is the per-surface activity log.

Edge functions

- **dialpad-initiate** (POST) — validates JWT in code via `requireAuth`, checks `requireIntegrationEnabled('dialpad')`, requires `staff.dialpad_user_id`. Calls `POST https://dialpad.com/api/v2/call` with `custom_data: { related_entity_type, related_entity_id, company_id }`, inserts `dialpad_calls` row (state `queued`), logs `outbound_call_initiated`, returns `{ dialpad_call_id }`.
- **dialpad-webhook** (public POST) — reads **raw body** and verifies `x-dialpad-signature = base64(HMAC-SHA256(raw, DIALPAD_WEBHOOK_SECRET))`; 401 on mismatch. Upserts `dialpad_calls` by `dialpad_call_id` (state, durations, URLs, transcript, timestamps, raw). When `custom_data` identifies a record, appends a `client_communications` row (type `phone`, direction matches; auto-summary like `Outbound call · 2:14 · Voicemail`). Inbound calls trigger screen-pop (see below). Logs `inbound_webhook` with `success/error`.
- **dialpad-backfill-user** (admin-only POST `{ staff_id, email }`) — calls `GET /api/v2/users?email=...`, writes numeric id to `staff.dialpad_user_id`.
- **dialpad-register-webhook** — one-shot helper to (re)register the company's webhook URL + signing secret with Dialpad.
- **dialpad-test-connection** — calls `GET /api/v2/users?limit=1`; on success stamps `last_connected_at` and returns user count + configured webhook URL. Registered with the Phase 11 hub.

All five Dialpad functions are declared in `supabase/config.toml` with `verify_jwt = false` and validate auth in code where needed.

Frontend

- `<CallButton phone relatedEntity={type,id} />` — invokes `dialpad-initiate`. Disabled when the integration is off or the current user has no `dialpad_user_id`. Wired into Client Detail, Lead Detail, Liability Detail (current creditor phone), Litigation Matter (opposing counsel phone), and Creditor Contact rows.
- **useInitiateCall hook** — `useMutation` wrapper with toast feedback.
- **Live call-state pill** — `useRealtimeSubscription` on `dialpad_calls` for `initiated_by = auth.uid()` (Queued → Ringing → Connected → Ended).
- **Activity entry on hangup** — activity stream renders the new `client_communications` row with duration, signed-URL recording link (reuses `<SignedDocumentLink>` pattern), and voicemail transcript.
- **Inbound screen-pop** — `ScreenPopProvider` subscribes to `dialpad_calls` inserts scoped to the current staff's company. Behavior driven by `staff.screen_pop_preference`:
 - `toast` (default) — sticky 30s toast with “Open” action.
 - `auto_navigate` — routes immediately to the matched record.
 - `off` — does nothing.

- **Staff → Dialpad mapping** — Staff Detail (admin) shows `dialpad_user_id` or a **Link to Dialpad** button invoking `dialpad-backfill-user`.

Phone matching

On inbound events the webhook normalizes `target_phone` to E.164 and looks up `clients` / `leads` / `creditor_contacts`. Single match → linked + screen-pop with `entity_type`, `entity_id`, `phone`. Multiple/no match → screen-pop with toast-only payload, no auto-navigation.

Acceptance

1. Admin can configure Dialpad on `/integrations`, run Test Connection, and see “Last connected”.
2. Staff with a linked `dialpad_user_id` see CallButtons; others see disabled state.
3. Click-to-call rings the staff’s Dialpad device; the live pill transitions through states.
4. Webhook signature mismatches are rejected (401) and logged with `success = false`.
5. Completed calls write to `client_communications` (client surfaces) or `entity_communications` (litigation, creditor, lead, and other polymorphic surfaces) with duration and (where present) signed recording link.
6. Inbound calls fire screen-pop per the staff member’s preference.
7. Disabling `dialpad` in `company_integrations` blocks `dialpad-initiate` immediately.
8. `dialpad_calls` is admin-readable within company scope; `service_role-only` writes.

Rollback (forward-only)

```
DROP TABLE IF EXISTS public.dialpad_calls;
ALTER TABLE public.staff DROP COLUMN IF EXISTS dialpad_user_id;
ALTER TABLE public.staff DROP COLUMN IF EXISTS screen_pop_preference;
DELETE FROM public.integration_providers WHERE provider_key = 'dialpad';
DELETE FROM public.company_integrations WHERE provider_key = 'dialpad';
```

Edge functions can be left deployed — they no-op once the registry row is gone.

Follow-ups

- Recording durability: currently we store the Dialpad URL and proxy via signed-link UI; option (b) — downloading into `client-documents` on webhook receipt — left as a future toggle.